📘 Post-Read Notes: Introduction to Apache Hive

These notes summarize all key ideas from the script in a clean, study-friendly format. They help you revise *why Hive exists, who it is for, how it works, and how it executes queries on Hadoop.*

1️⃣ Recap: Why Hadoop & MapReduce?

- Hadoop was created to store & process **big data at scale**
- MapReduce enables **distributed processing**
- Writing MapReduce requires **strong programming skills (mainly Java)**
- Processed data is stored back in **HDFS**

👉 Problem: Not everyone working with data is a programmer.

2️⃣ Do You Need Programming for Big Data?

Reality

Many industry users:

- Are analysts, not developers
- Want insights, not code
- Struggle with Java-heavy MapReduce

Hadoop Internals

- Hadoop core → written in Java

- Some performance parts → C/C++ (compression, etc.)

➡️ This created a usability bottleneck.

3️⃣ Target Persona Concept

What is a Persona?

A persona = fictional but realistic representation of a user group.

Used to design:

- Products
- Courses
- Tools

Example: Students with different tech backgrounds.

4️⃣ MapReduce vs Hive Personas

👨‍💻 MapReduce Persona

Who?

- Developers
- Data Engineers

Characteristics

- Comfortable coding
- Prefer control over simplicity
- Fine with long programs

Mindset

> Coding is a strength, not a barrier.

Best For

- Complex logic
- Custom processing
- Full control workflows

📊 Hive Persona

Who?

- Analysts
- Business users
- Students
- Non-engineers

Characteristics

- Strong in business thinking
- SQL familiarity
- Focused on insights

Mindset

> “Tell me what the data says.”

Best For

- Analytics
- Reporting
- Trend analysis

5️⃣ MapReduce vs Hive (Key Differences)

Aspect	MapReduce	Hive
Nature	Low-level framework	High-level warehouse
Programming	Required	Not required
Language	Java/APIs	HiveQL (SQL-like)
Users	Developers	Analysts
Learning	Hard	Easy
Abstraction	Low	High
Query style	Code	Declarative SQL
Dev time	High	Low
Productivity	Lower	Higher
Use case	Complex logic	Analytics
Execution	Direct MR	Converts to MR/Tez/Spark
Schema	None	Schema-on-read
Metadata	None	Hive Metastore
Maintenance	High	Low

Flexibility	Very high	Moderate
Tuning	Manual	Mostly automatic
Debugging	Hard	Easier
Real-time	No	No
Role	Base layer	Built on MR

---

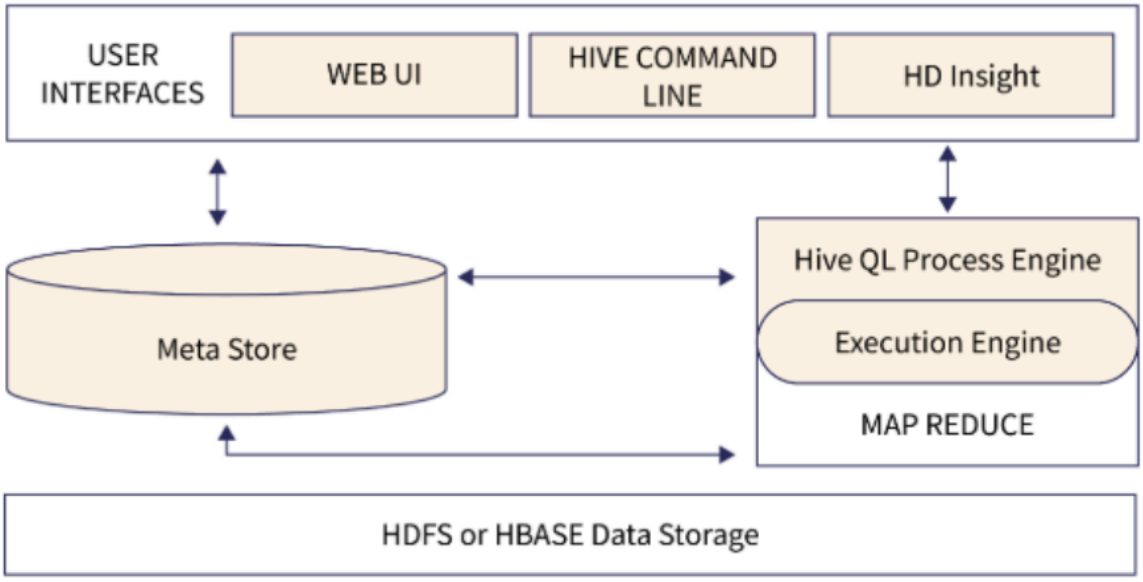
6 Basic Working Principle of Hive

Flow

1. User writes HiveQL
2. Hive parses & validates
3. Query → execution steps
4. Runs on Hadoop
5. Results returned or stored

---

7 Hive Architecture



🔹 User Interface

Hive CLI (Deprecated)

- Original terminal interface
- Being replaced

Beeline (Preferred)

- JDBC-based client
- Multi-user support
- Secure & scalable

Hue

- Web UI
- SQL editor in browser
- No command line needed

JDBC/ODBC

- Connect BI tools (Tableau, Power BI)
- External app integration

🔹 Driver

Acts as coordinator:

- Manages full query lifecycle
- Parsing → compiling → execution → results
- Coordinates with execution engines

🔹 Compiler

Responsibilities

- Convert HiveQL → execution plan
- Break query into logical steps
- Create physical plan
- Apply optimizations

Validations

- Syntax checks
- Semantic checks
- Table/column validation
- Data type checks
- Function validation

8️⃣ Hive Metastore ⭐ (Very Important)

What is it?

Central metadata repository.

Stores:

- Table schemas
- Columns
- Partitions
- Storage locations
- Databases

❗ Does NOT store actual data.

Analogy

Teacher's register:

- Register = metadata

- Students = actual data

Hive Metastore = register for data.

9️⃣ Hive Databases

- Logical namespaces
- Organize tables/views
- Map to directories in HDFS/cloud

🔟 Hive Tables

- Structured view of data
- Define schema
- Specify format & location
- Allow SQL querying

1️⃣1️⃣ Columns

- Define data structure
- Stored as metadata
- Actual values stored in files

- Enable filtering & analysis

1️⃣2️⃣ Partitions ⭐

What?

Split table by column values.

Each partition → separate directory.

Benefit

Partition pruning:

👉 Hive scans only required data.

Example

```
employee/join_date=2008-09-01/
```

Query on that date → reads only that folder.

Analogy

School sections:

Check attendance for one class → not whole school.

1️⃣3️⃣ Bucketing

What?

Further divide data using hash function.

Each bucket → separate file.

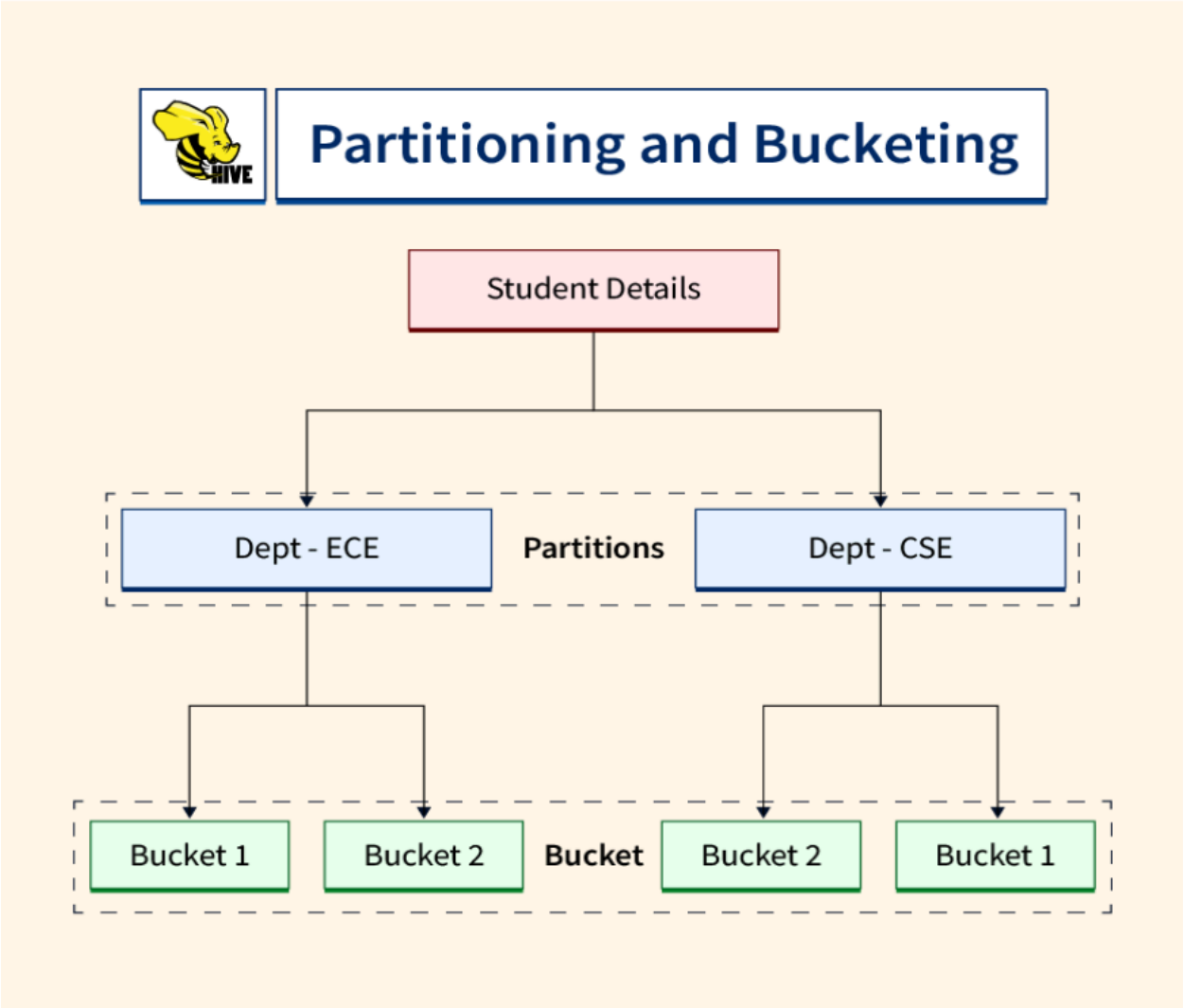
Benefits

- Faster joins
- Efficient sampling
- Even data distribution

Analogy

Students grouped by roll numbers.

Predictable grouping.



1️⃣4️⃣ Execution Engine

Hive doesn’t execute directly — it translates.

🔹 MapReduce (Legacy)

Process

HiveQL → MR jobs → YARN execution.

Characteristics

- Disk heavy
- Slower
- Reliable

Analogy

Teachers checking papers manually.

🔹 Tez & Spark (Modern)

Features

- DAG execution
- In-memory processing
- Reduced disk I/O
- Faster queries

Analogy

Digital evaluation system.

1️⃣5️⃣ Choosing Execution Engine

Early Hive → MapReduce only.

Now:

- **Tez = default**
- Faster DAG model
- Better resource usage

Set Engine

```
SET hive.execution.engine=tez;
SET hive.execution.engine=mr;
SET hive.execution.engine=spark;
```

Check Engine

```
SET hive.execution.engine;
```

Important Notes

- HiveQL stays same
- Only execution changes
- Tez/Spark need setup
- Hive may fallback if misconfigured

---

16 Engine Comparison

Feature	MR	Tez	Spark
Model	Map→Reduce	DAG	DAG
Default	Hive 1.x	Hive 2.x+	Optional

Performance	Slow	Fast	Fastest
Disk I/O	High	Reduced	Minimal
Latency	High	Medium-Low	Low
Use Case	Batch	Analytics	ML/Streaming
Efficiency	Low	Good	Very High

1️⃣7️⃣ Hive Table Creation Examples

Basic Managed Table

```
CREATE TABLE sales (
  order_id INT,
  customer_id INT,
  amount DOUBLE
);
```

Hive owns data → drop table = delete data.

Add Delimiter

```
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

Correct CSV parsing.

Define File Type

```
STORED AS TEXTFILE;
```

Clear storage format.

Add Partitioning

```
PARTITIONED BY (order_date STRING)
```

Improves performance.

Add Bucketing

```
CLUSTERED BY (customer_id) INTO 4 BUCKETS
```

Optimizes joins & aggregations.

✅ Key Takeaway

Even after adding:

- Partitioning
- Bucketing
- Storage formats

👉 **HiveQL for users stays the same.**

These settings only change:

- Storage layout
- Processing efficiency
- Query performance

🧠 Final Big Picture

Hive makes Hadoop usable for non-programmers.

It provides:

- SQL interface on big data
- Metadata management
- Performance optimizations
- Execution engine abstraction

👉 Hive = Bridge between Hadoop power & SQL simplicity.